WeatherWise

Plateforme intelligente de suivi météorologique



7	8	6
microservices	bases MySQL	membres

Notre équipe



Khawla Najai	Nadine Rahal	Sara jerbi	Ghassen Denguir	Aziz Sakouhi	Wissem
Membre 1	Membre 2	Membre 3	Membre 4	Membre 5	Membre 6
Weather-Data Service	Forecast Service	Weather-Alert Service	User-Preferences Service	City & Favorites Service	Trip-Planner Service

Composants collectifs : configuration de Keycloak et développement du Weather-Insights-Service (Django)

Sommaire



Idée & problématique

Acteurs, cas d'utilisation, fonctionnalités



Architecture globale

Gateway, sécurité, données, communications



Microservices

Conception UML, interactions et contrats d'API



Technologies

Les choix techniques de la plateforme



Organisation

Tâches, dépôt Git, outil de gestion



Planning & risques

Calendrier sur 4 semaines, conclusion

L'idée et la problématique

Idée générale

Application web de suivi météo : météo actuelle, prévisions, villes favorites, préférences, alertes par e-mail, planification de voyages et recommandations selon les conditions prévues.

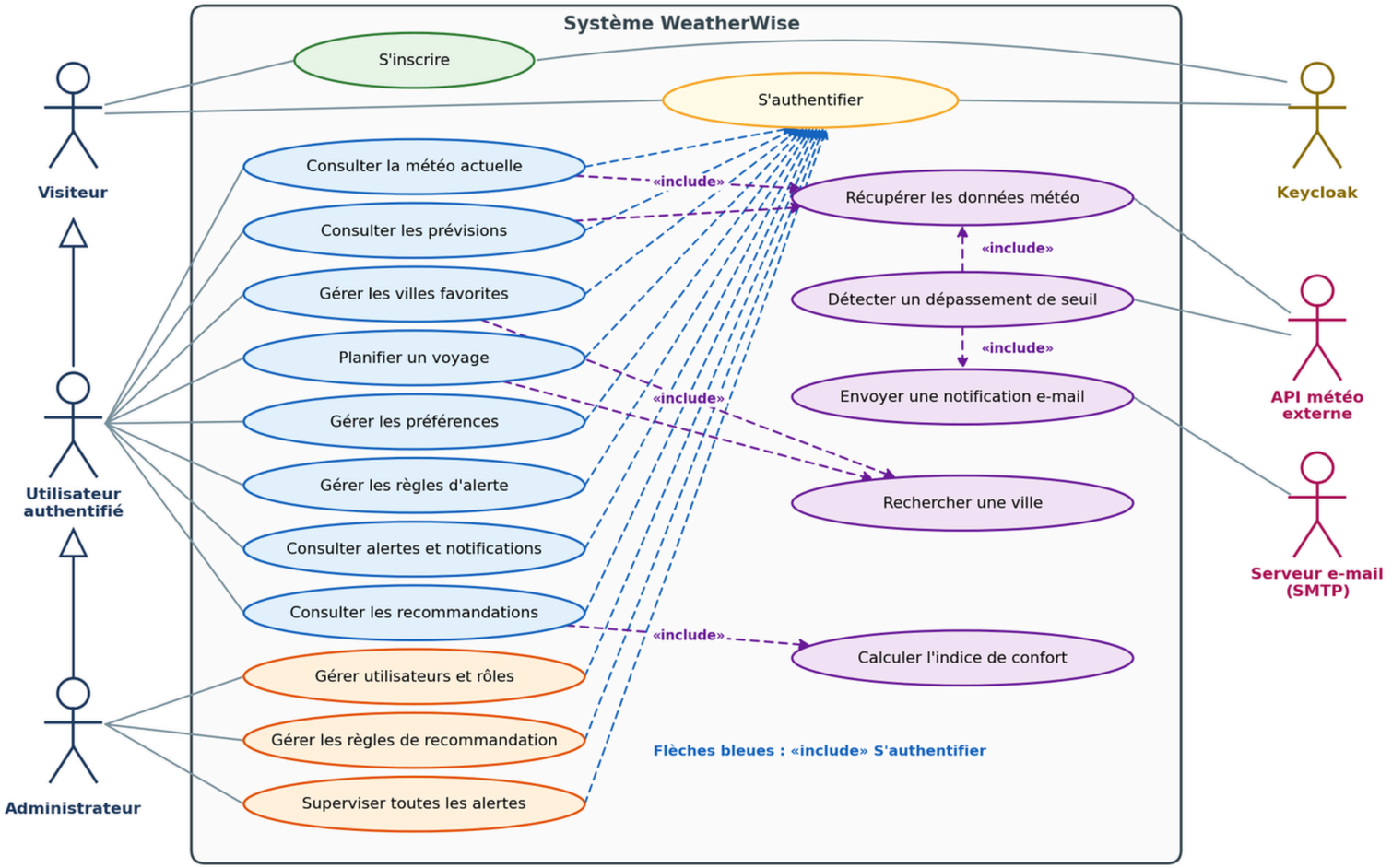
Problématique

Comment concevoir une plateforme météo modulaire, personnalisée, capable de détecter les conditions à risque et d'aider à planifier ses déplacements, tout en garantissant l'indépendance des services et la sécurité des accès ?

Objectifs du projet

- Consulter la météo actuelle et les prévisions horaires / journalières
- Gérer villes favorites et préférences (unités, langue, notifications)
- Détecter les seuils dépassés et notifier par e-mail
- Planifier des voyages avec un résumé météo par étape
- Produire un indice de confort et des recommandations
- Centraliser l'authentification avec Keycloak
- Garantir sécurité, maintenabilité et évolutivité

Acteurs et cas d'utilisation



Visiteur

S'inscrit et s'authentifie

Utilisateur

Météo, favoris, préférences, alertes, voyages

Administrateur

Utilisateurs, rôles, règles de recommandation, supervision

Keycloak

Serveur d'identité (acteur secondaire)

API météo externe

Observations et prévisions

Serveur SMTP

Acheminement des e-mails d'alerte

Fonctionnalités principales

Authentification

Inscription, connexion, rôles USER / ADMIN via Keycloak

Météo actuelle

Température, humidité, pression, vent

Prévisions

Horaires et journalières, comparaison de périodes

Villes & favoris

Recherche de villes et liste de favoris

Alertes

Règles à seuils, niveaux de gravité, états

Notifications e-mail

Envoi SMTP et historique des envois

Préférences

Unités, langue, canal de notification

Voyages

Étapes, prévisions et risque par étape

Recommandations

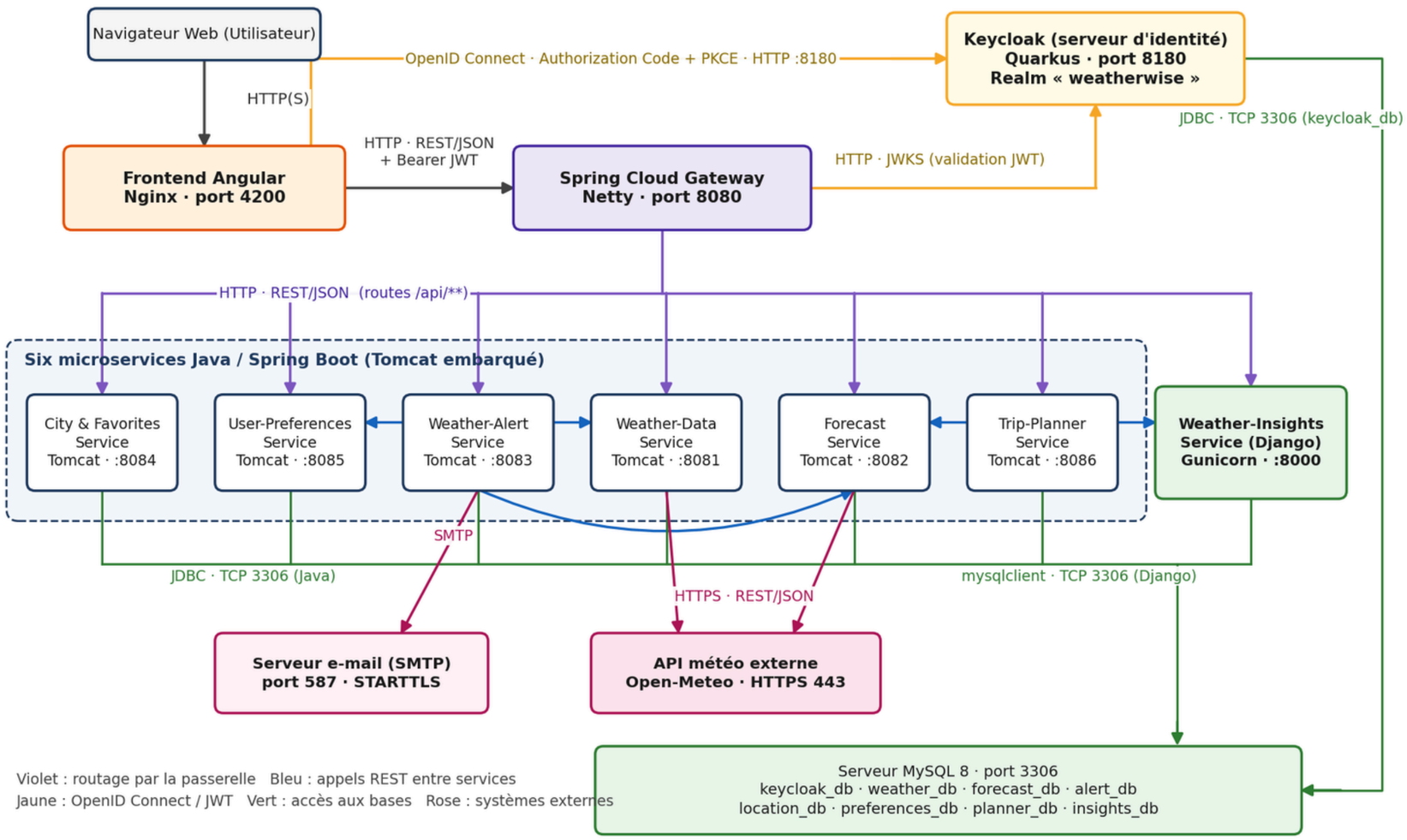
Indice de confort et conseils basés sur des règles

01

Architecture globale

Diagramme global, API Gateway, sécurité, données et communications

Architecture logique globale



Frontend Angular

Nginx · port 4200

Spring Cloud Gateway

Point d'entrée unique · 8080

Keycloak

OIDC / OAuth2 / JWT · 8180

MySQL 8

8 bases indépendantes · 3306

API Gateway : Spring Cloud Gateway



Rôle de la passerelle

- Point d'entrée unique du backend
- Validation du JWT (JWKS Keycloak)
- Routage par préfixe /api/…
- Contrôle du rôle requis par route

Route exposée	Service cible	Port	Rôle requis
/api/weather/**	Weather-Data-Service	8081	USER
/api/forecast/**	Forecast-Service	8082	USER
/api/alerts/**	Weather-Alert-Service	8083	USER · ADMIN (supervision)
/api/cities/**	City & Favorites-Service	8084	USER
/api/preferences/**	User-Preferences-Service	8085	USER
/api/trips/**	Trip-Planner-Service	8086	USER
/api/insights/**	Weather-Insights-Service	8000	USER · ADMIN (règles)

Sécurité et authentification

1. Le frontend Angular redirige l'utilisateur vers Keycloak
2. Keycloak authentifie (Authorization Code + PKCE) et renvoie un JWT
3. Le frontend appelle la passerelle avec Authorization: Bearer
4. La passerelle valide signature et durée via les clés publiques (JWKS)
5. Chaque service contrôle le rôle (USER / ADMIN) et lit l'identifiant sub

Configuration Keycloak

- Realm weatherwise, rôles USER et ADMIN
- Client public weatherwise-frontend (PKCE)
- Gateway et services : resource servers
- Base dédiée keycloak_db
- Realm exporté en JSON, sans secret, versionné dans Git

Données : une base par service



keycloak_db

Keycloak

Comptes, rôles, sessions, realm



weather_db

Weather-Data

Observations météorologiques



forecast_db

Forecast

Prévisions et périodes



alert_db

Weather-Alert

Règles, alertes, notifications



location_db

City & Favorites

Villes et favoris



preferences_db

User-Preferences

Préférences des utilisateurs



planner_db

Trip-Planner

Voyages et étapes



insights_db

Weather-Insights

Règles et recommandations

Aucune clé étrangère entre bases : l'utilisateur est référencé par le claim sub du JWT Keycloak

Communications entre services

SYNCHRONE

Résumé météo d'un voyage (REST / JSON)

Angular
GET /api/trips/{id}/summary + JWT

→

Gateway
Valide le jeton, route vers le service

→

Trip-Planner
Vérifie le propriétaire du voyage

→

Forecast
Prévisions de chaque étape

→

Insights
Risque et recommandations

ASYNCHRONE

Alerte météo → notification e-mail

Weather-Alert
Détecte un seuil dépassé

→

Préférences
Lit le canal et l'activation

→

SMTP
Envoi de l'e-mail (STARTTLS)

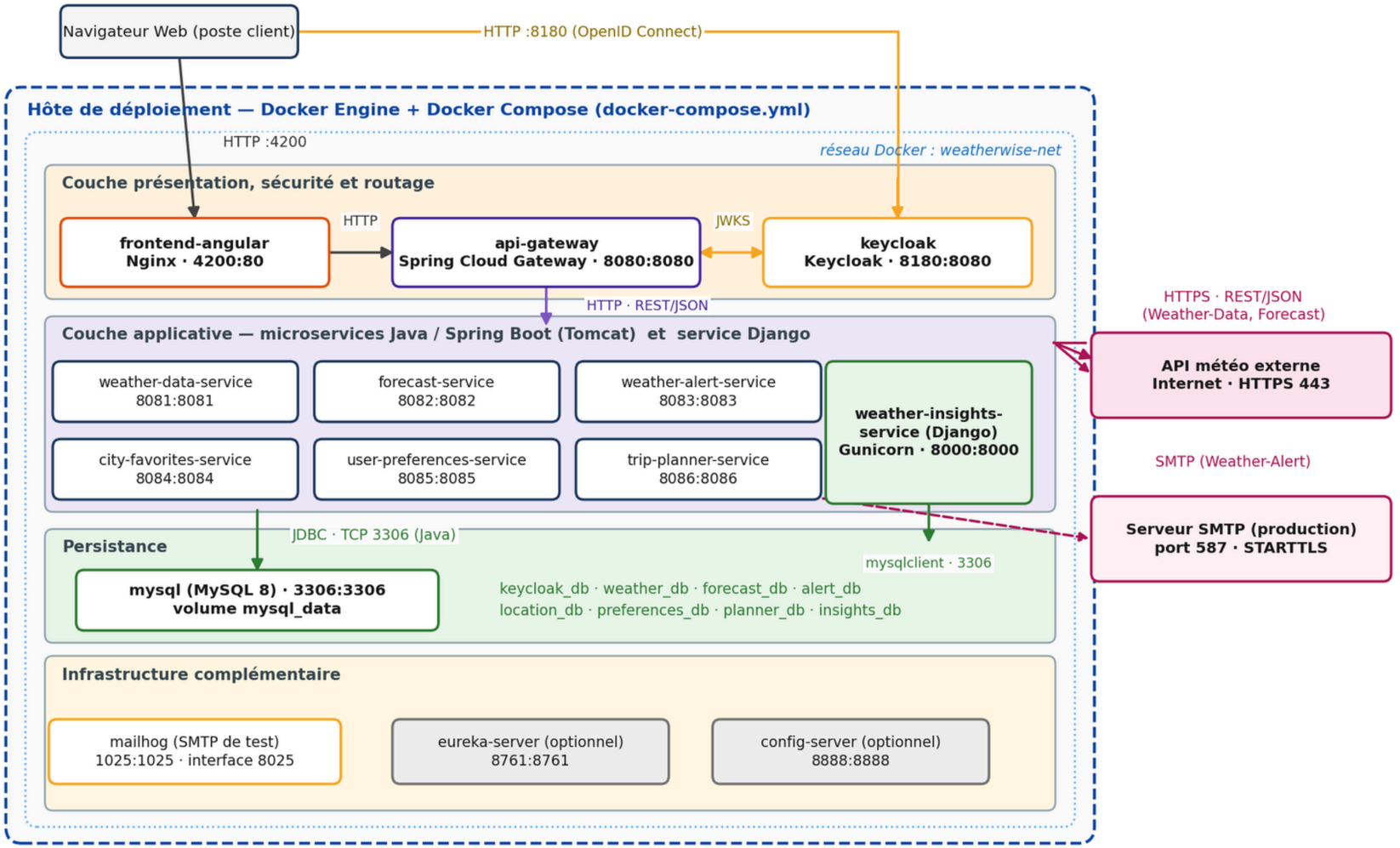
→

alert_db
Statut PENDING, SENT ou FAILED

→

Utilisateur
Reçoit l'e-mail, voit l'historique

Architecture physique et déploiement



Docker Compose

Conteneurs sur un réseau Docker commun

Un serveur MySQL

Huit bases séparées

MailHog en test

SMTP réel en production

02

Les microservices

Responsabilités, conception UML, interactions et contrats d'API

Vue d'ensemble des services

Porteur	Service	Technologies	Port	Base	Fonction principale
Membre 1	**Weather-Data-Service**	Java · Spring Boot · JPA	8081	weather_db	Données météo actuelles et historique
Membre 2	**Forecast-Service**	Java · Spring Boot · JPA	8082	forecast_db	Prévisions horaires et journalières
Membre 3	**Weather-Alert-Service**	Java · Spring Boot · JPA · Spring Mail	8083	alert_db	Alertes et envoi des notifications e-mail
Membre 4	**User-Preferences-Service**	Java · Spring Boot · Spring Security	8085	preferences_db	Profils et préférences des utilisateurs
Membre 5	**City & Favorites-Service**	Java · Spring Boot · JPA	8084	location_db	Recherche de villes et favoris
Membre 6	**Trip-Planner-Service**	Java · Spring Boot · JPA	8086	planner_db	Planification de voyages selon la météo
Équipe	**Weather-Insights-Service**	Python · Django · DRF · Gunicorn	8000	insights_db	Indice de confort, risque et recommandations
Équipe	**Keycloak**	Serveur d'identité	8180	keycloak_db	Comptes, rôles, jetons JWT

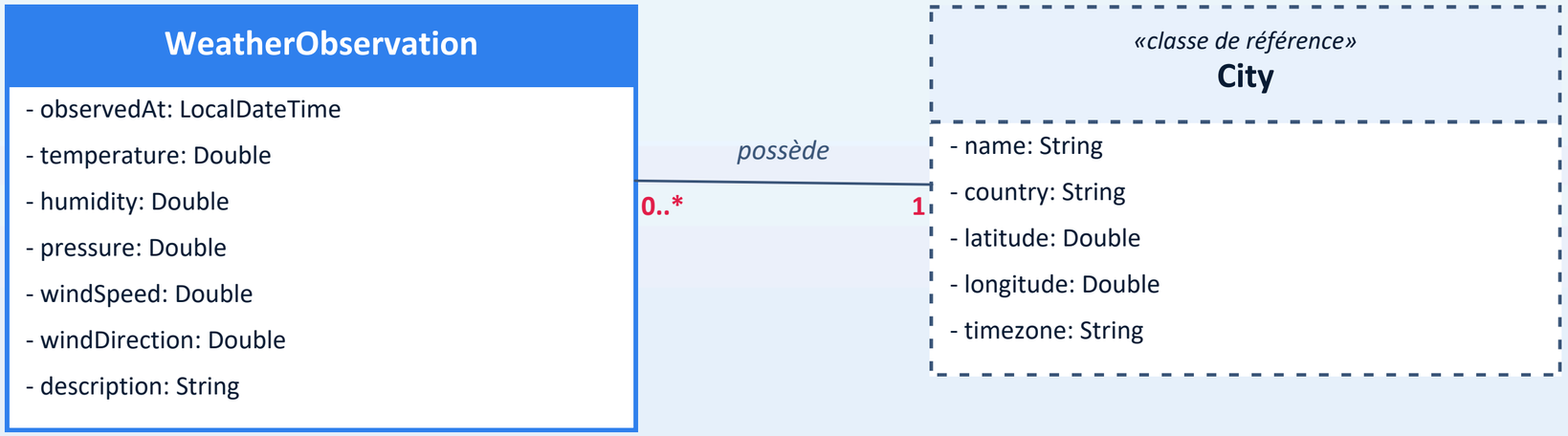
Conventions des contrats d'API

200	**OK** Lecture ou modification réussie	201	**Created** Ressource créée
202	**Accepted** Traitement lancé	204	**No Content** Suppression réussie
400	**Bad Request** Données invalides	401	**Unauthorized** JWT absent ou invalide
403	**Forbidden** Rôle ou propriétaire incorrect	404	**Not Found** Ressource introuvable
409	**Conflict** Doublon ou dates incohérentes	503	**Unavailable** API météo ou service injoignable

Règles communes

- Échanges en JSON sur HTTP
- Authorization: Bearer <JWT>
- Contrats et schémas JSON définis avant l'intégration
- Documentation Swagger / OpenAPI
- Événements : évolution proposée

Weather-Data-Service · Conception



Classes en pointillés : classes d'un autre composant, liées par appels REST (pas de jointure entre bases)

Membre 1 · weather_db

Synchrone (REST / JSON)

- Frontend → Gateway → Weather-Data : GET /api/weather/current?city=Tunis
- Weather-Data → Open-Meteo : GET (HTTPS), réponse mise en cache
- Alert-Service → Weather-Data : GET /api/weather/current (seuils)

Asynchrone

- Job planifié : actualisation périodique des observations
- Événement proposé : WeatherObservationUpdated

Weather-Data-Service · Contrat d'API

Méthode	Endpoint	Description	Codes de réponse
GET	`/api/weather/current?city={name}`	Météo actuelle d'une ville	200 · 400 · 404 · 503
GET	`/api/weather/current?lat={}&lon={}`	Météo actuelle par coordonnées	200 · 400 · 503
GET	`/api/weather/history?city=&from=&to=`	Historique des observations	200 · 400 · 404
GET	`/api/weather/observations/{id}`	Détail d'une observation	200 · 404
POST	`/api/weather/refresh`	Forcer l'actualisation (ADMIN)	202 · 401 · 403

Sécurité

USER pour la consultation · ADMIN pour l'actualisation forcée

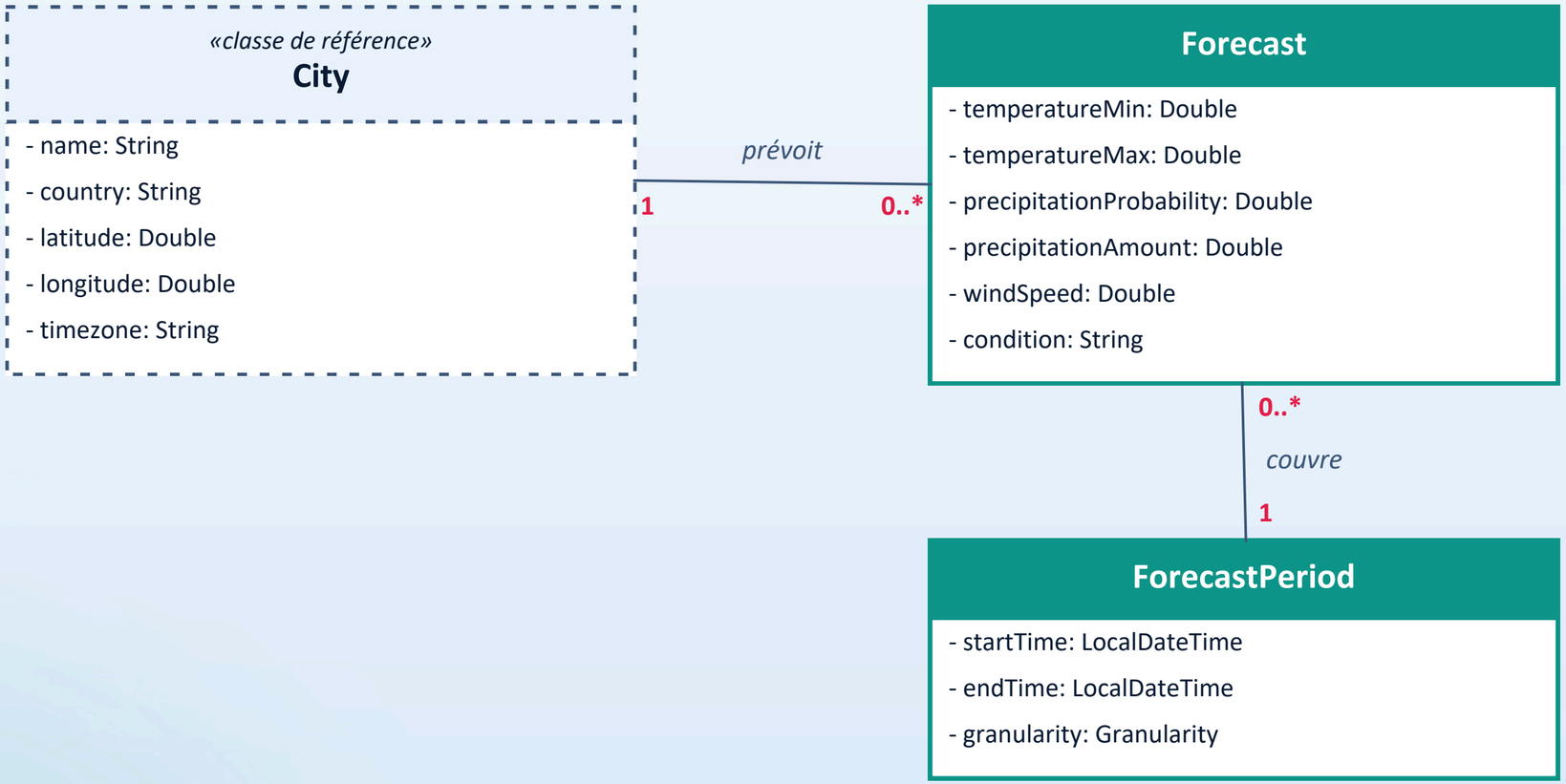
Événements publiés

WeatherObservationUpdated (proposé)

Événements consommés

Aucun : source des données

Forecast-Service · Conception



Classes en pointillés : classes d'un autre composant, liées par appels REST (pas de jointure entre bases)

Membre 2 · forecast_db

Synchrone (REST / JSON)

- Trip-Planner → Forecast : GET /api/forecast/daily?city=&days=5
- Alert-Service → Forecast : GET /api/forecast/hourly?city=
- Forecast → Open-Meteo : GET (HTTPS) pour actualiser

Asynchrone

- Job planifié : actualisation des prévisions
- Événement proposé : ForecastUpdated

Forecast-Service · Contrat d'API

Méthode	Endpoint	Description	Codes de réponse
GET	`/api/forecast/hourly?city=&from=&to=`	Prévisions horaires	200 · 400 · 404
GET	`/api/forecast/daily?city=&days=7`	Prévisions journalières	200 · 400 · 404
GET	`/api/forecast/compare?city=&a=&b=`	Comparer deux périodes	200 · 400 · 404
POST	`/api/forecast/refresh`	Actualiser les prévisions (ADMIN)	202 · 401 · 403

Sécurité

USER pour la consultation · ADMIN pour l'actualisation

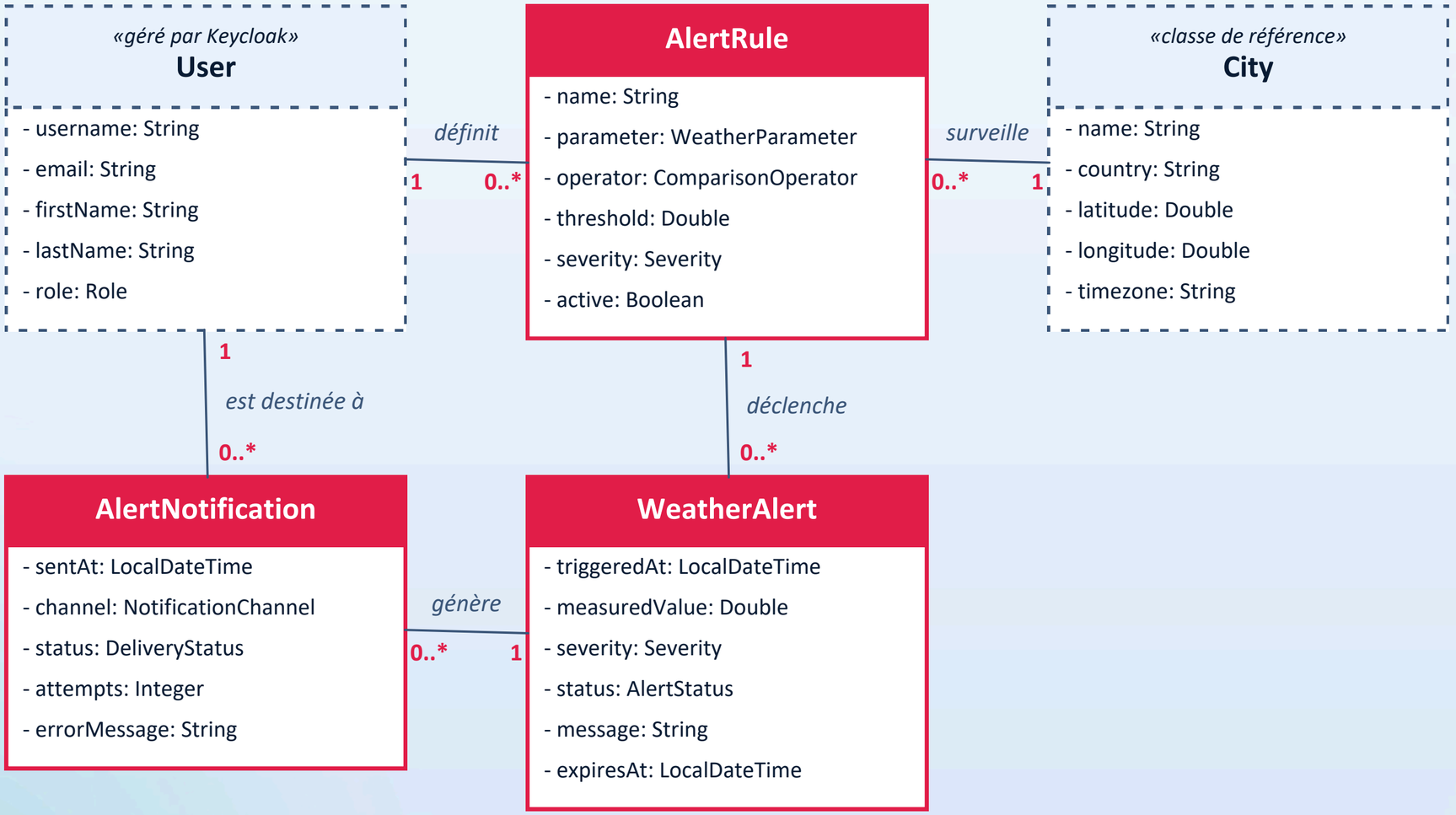
Événements publiés

ForecastUpdated (proposé)

Événements consommés

Aucun : source des prévisions

Weather-Alert-Service · Conception



Classes en pointillés : classes d'un autre composant, liées par appels REST (pas de jointure entre bases)

Membre 3 · alert_db

Synchrone (REST / JSON)

- Alert → Weather-Data : GET /api/weather/current?city=
- Alert → Forecast : GET /api/forecast/hourly?city=
- Alert → User-Preferences : GET /api/preferences/{userId}

Asynchrone

- E-mail via SMTP : statut PENDING → SENT ou FAILED, nouvel essai si échec
- Événement proposé : AlertTriggered

Weather-Alert-Service · Contrat d'API

Méthode	Endpoint	Description	Codes de réponse
POST	`/api/alerts/rules`	Créer une règle d'alerte	201 · 400 · 401
GET	`/api/alerts/rules`	Lister mes règles	200 · 401
PUT	`/api/alerts/rules/{id}`	Modifier une règle	200 · 400 · 403 · 404
DELETE	`/api/alerts/rules/{id}`	Supprimer une règle	204 · 403 · 404
GET	`/api/alerts`	Alertes détectées pour moi	200 · 401
GET	`/api/alerts/notifications`	Historique des notifications	200 · 401
GET	`/api/alerts/admin/all`	Supervision globale (ADMIN)	200 · 403

Sécurité

USER sur ses propres règles · ADMIN pour la supervision globale

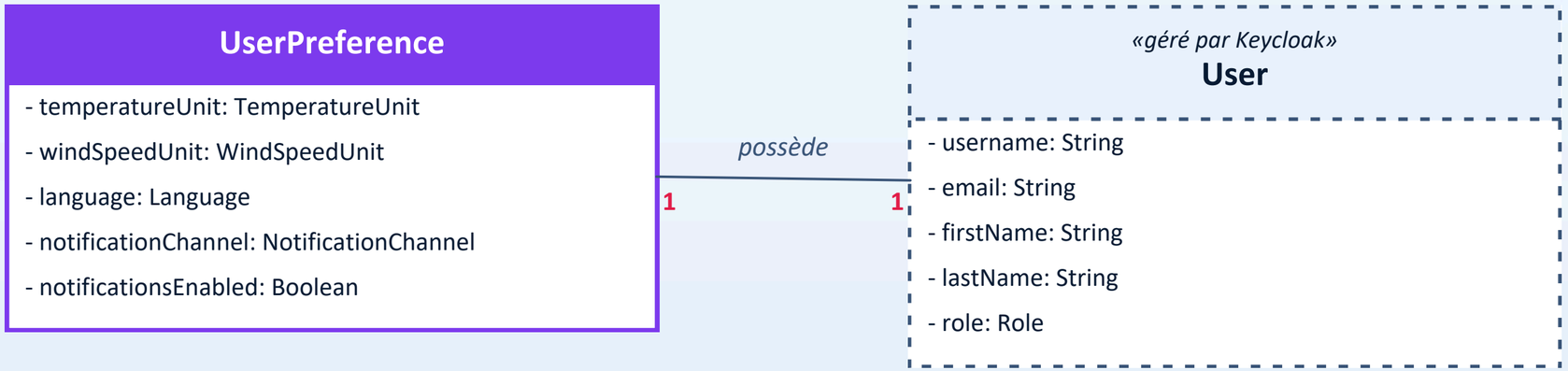
Événements publiés

AlertTriggered · NotificationSent · NotificationFailed (proposés)

Événements consommés

PreferencesUpdated (proposé)

User-Preferences-Service · Conception



Classes en pointillés : classes d'un autre composant, liées par appels REST (pas de jointure entre bases)

Membre 4 · preferences_db

Synchrone (REST / JSON)

- Frontend → Gateway → Preferences : PUT /api/preferences/me
- Alert → Preferences : GET /api/preferences/{userId}
- Preferences → Keycloak : validation JWT via JWKS

Asynchrone

- Profil par défaut créé à la première connexion
- Événement proposé : PreferencesUpdated

User-Preferences-Service · Contrat d'API

Méthode	Endpoint	Description	Codes de réponse
GET	`/api/preferences/me`	Lire mes préférences (profil créé si absent)	200 · 401
PUT	`/api/preferences/me`	Remplacer mes préférences	200 · 400 · 401
PATCH	`/api/preferences/me/notifications`	Activer / couper les notifications	200 · 400 · 401
GET	`/api/preferences/{userId}`	Lecture inter-services (ADMIN / service)	200 · 403 · 404
DELETE	`/api/preferences/me`	Réinitialiser mon profil	204 · 401

Sécurité

USER sur son profil (claim sub du JWT) · ADMIN pour la lecture d'un autre profil

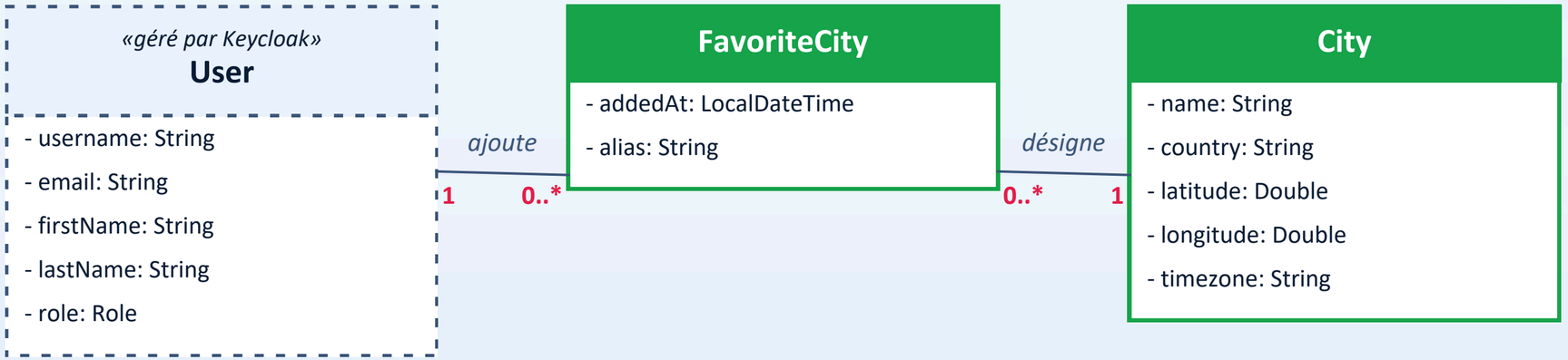
Événements publiés

PreferencesUpdated (proposé)

Événements consommés

Aucun

City & Favorites-Service · Conception



Classes en pointillés : classes d'un autre composant, liées par appels REST (pas de jointure entre bases)

Membre 5 · location_db

Synchrone (REST / JSON)

- Frontend → Gateway → City : GET /api/cities/search?name=Tunis
- Weather-Data → City : GET /api/cities/{id} (coordonnées)
- Trip-Planner → City : GET /api/cities/{id}

Asynchrone

- Événement proposé : FavoriteAdded
- Utilisation : préchargement de la météo des favoris

City & Favorites-Service · Contrat d'API

Méthode	Endpoint	Description	Codes de réponse
GET	`/api/cities/search?name={q}`	Rechercher une ville par nom	200 · 400
GET	`/api/cities/search?lat=&lon=`	Rechercher par coordonnées	200 · 400 · 404
GET	`/api/cities/{id}`	Détail d'une ville	200 · 404
GET	`/api/cities/favorites`	Lister mes villes favorites	200 · 401
POST	`/api/cities/favorites`	Ajouter un favori	201 · 400 · 401 · 409
DELETE	`/api/cities/favorites/{id}`	Retirer un favori	204 · 401 · 404

Sécurité

USER ; favoris liés à l'identifiant Keycloak (claim sub) ; 409 si doublon

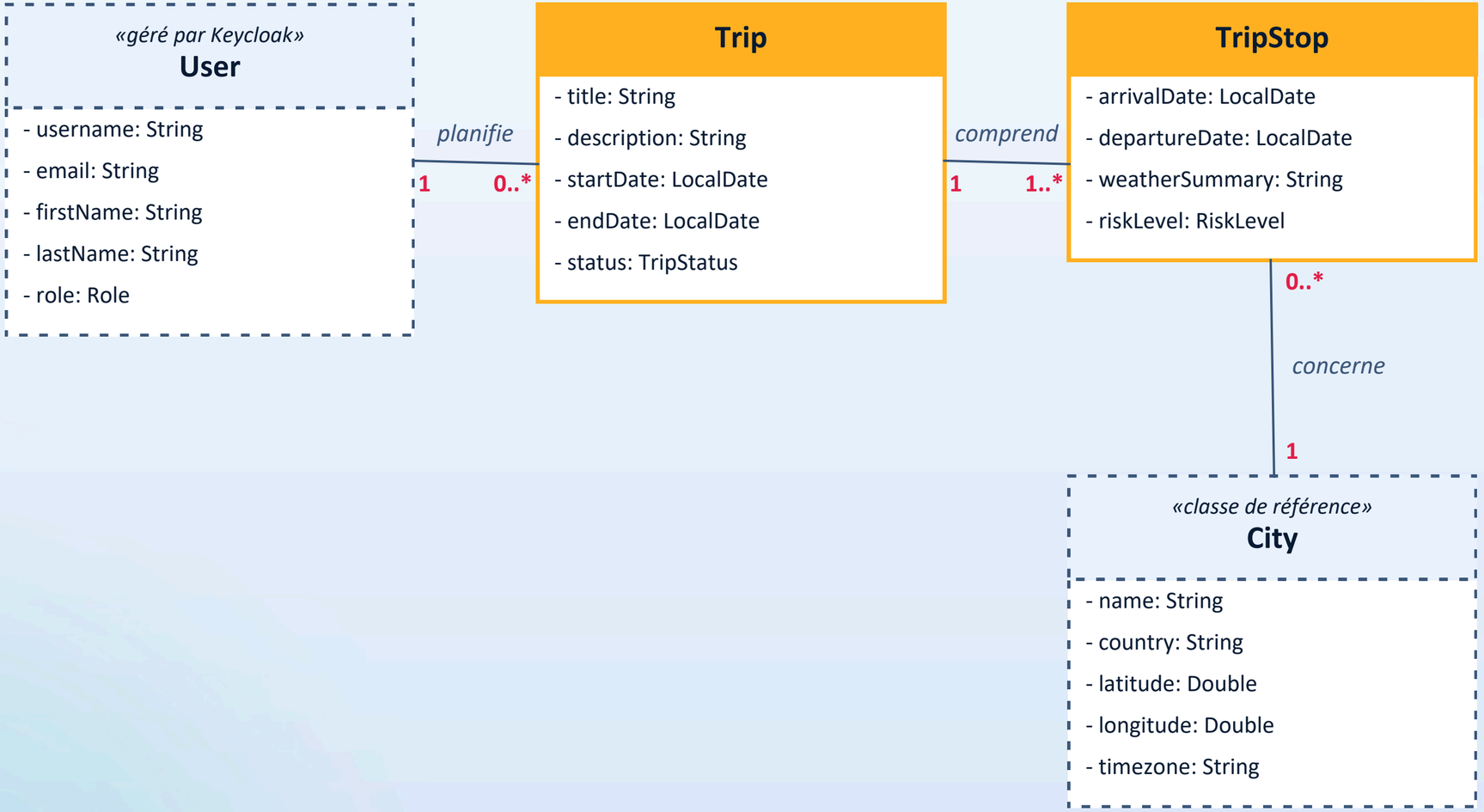
Événements publiés

FavoriteAdded · FavoriteRemoved (proposés)

Événements consommés

Aucun

Trip-Planner-Service · Conception



Classes en pointillés : classes d'un autre composant, liées par appels REST (pas de jointure entre bases)

Membre 6 · planner_db

Synchrone (REST / JSON)

- Trip → Forecast : GET /api/forecast/daily?city=&days= par étape
- Trip → Insights : POST /api/insights/recommendations
- Trip → City : GET /api/cities/{id}

Asynchrone

- Événement proposé : TripStopAdded
- Utilisation : recalcul du niveau de risque d'une étape

Trip-Planner-Service · Contrat d'API

Méthode	Endpoint	Description	Codes de réponse
POST	`/api/trips`	Créer un voyage	201 · 400 · 401
GET	`/api/trips`	Lister mes voyages	200 · 401
GET	`/api/trips/{id}`	Détail d'un voyage	200 · 403 · 404
PUT	`/api/trips/{id}`	Modifier un voyage	200 · 400 · 403 · 404
DELETE	`/api/trips/{id}`	Supprimer un voyage	204 · 403 · 404
POST	`/api/trips/{id}/stops`	Ajouter une étape	201 · 400 · 403 · 404 · 409
GET	`/api/trips/{id}/summary`	Résumé météo, risque, recommandations	200 · 403 · 404 · 503

Sécurité

USER propriétaire du voyage uniquement (403 sinon)

Événements publiés

TripCreated · TripStopAdded (proposés)

Événements consommés

ForecastUpdated (proposé)

Weather-Insights-Service · Conception



Classes en pointillés : classes d'un autre composant, liées par appels REST (pas de jointure entre bases)

Équipe · insights_db

Synchrone (REST / JSON)

- Trip → Insights : POST /api/insights/recommendations
- Frontend → Gateway → Insights : GET /api/insights/recommendations?city=
- Insights → Keycloak : validation JWT (JWKS)

Asynchrone

- Événement proposé : RuleUpdated (règle modifiée par un ADMIN)
- Utilisation : invalidation du cache des recommandations

Weather-Insights-Service · Contrat d'API

Méthode	Endpoint	Description	Codes de réponse
POST	`/api/insights/comfort`	Calculer l'indice de confort	200 · 400 · 401
POST	`/api/insights/risk`	Déterminer le niveau de risque	200 · 400 · 401
GET	`/api/insights/recommendations?city=`	Recommandations pour une ville	200 · 401 · 404
GET	`/api/insights/rules`	Lister les règles (ADMIN)	200 · 403
POST	`/api/insights/rules`	Créer une règle (ADMIN)	201 · 400 · 403
PUT	`/api/insights/rules/{id}`	Modifier une règle (ADMIN)	200 · 400 · 403 · 404

Sécurité

USER pour les calculs et recommandations · ADMIN pour les règles

Événements publiés

RuleUpdated (proposé)

Événements consommés

Aucun

03

Technologies et organisation

Choix techniques, répartition des tâches, Git, planning et risques

Technologies choisies



Backend

- Java · Spring Boot
- Spring Data JPA / Hibernate
- Spring Cloud Gateway
- Eureka, Config Server (si retenus)



Service commun

- Python · Django
- Django REST Framework
- Gunicorn · Django ORM



Sécurité

- Keycloak · OAuth2 · OpenID Connect
- JWT et JWKS
- Spring Security (resource server)



Données et intégrations

- MySQL 8, une base par service
- Open-Meteo ou équivalent
- SMTP (Spring Mail), MailHog en test



Frontend

- Angular
- Nginx (conteneur)
- Flux Authorization Code + PKCE



Outils et qualité

- Docker · Docker Compose
- Git · GitHub · Trello ou Jira
- JUnit · Mockito · Postman
- Swagger / OpenAPI

Répartition des tâches

1

Membre 1

Weather-Data-Service

Contribution transverse

Contrats d'API et intégration de la passerelle

2

Membre 2

Forecast-Service

Contribution transverse

Intégration de l'API météo et tests des prévisions

3

Membre 3

Weather-Alert-Service

Contribution transverse

Serveur SMTP et tests des alertes

4

Membre 4

User-Preferences-Service

Contribution transverse

Configuration de Keycloak et sécurité globale

5

Membre 5

City & Favorites-Service

Contribution transverse

Frontend de recherche et de favoris

6

Membre 6

Trip-Planner-Service

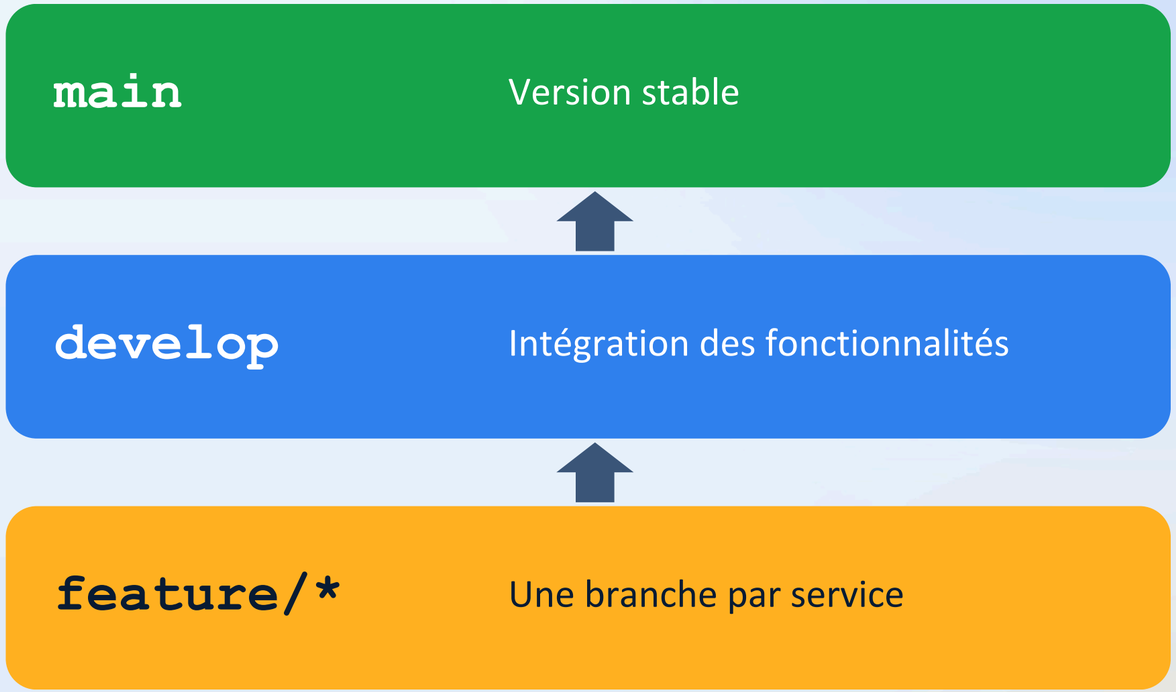
Contribution transverse

Frontend de planification et tableau de bord

Collectif : Weather-Insights-Service (Django) et configuration Keycloak, répartis par modules

Organisation du dépôt Git

```
WeatherWise/
├── Backend/
│   ├── weather-data-service/
│   ├── forecast-service/
│   ├── weather-alert-service/
│   ├── city-favorites-service/
│   ├── user-preferences-service/
│   ├── trip-planner-service/
│   ├── weather-insights-service/
│   ├── api-gateway/
│   ├── keycloak/  (realm, sans secret)
│   └── docker-compose.yml
├── Frontend/
├── Documentation/
└── README.md
```



Règles de collaboration

- Pull Request, revue de code et tests avant fusion
- Aucun secret publié : fichier .env.example
- Lancement documenté dans le README

Outil de gestion de projet

À faire	En cours	En revue	Terminé
Définir les contrats d'API	Développer les entités et services	Pull Request en relecture	Tests validés, fusionné dans develop
…	…	…	…



Issues GitHub

Une issue par tâche importante



Réunion hebdomadaire

Avancement, blocages, intégration



Définition de « terminé »

Code relu, tests passés, documenté

Planning prévisionnel

Semaine 1 · Cadrage

- Architecture et responsabilités
- Contrats REST et UML
- Keycloak, dépôt GitHub

Résultat attendu

Architecture validée

Semaine 2 · Services Java

- Entités et repositories
- Services et contrôleurs
- Tests unitaires

Résultat attendu

Premières API et persistance MySQL

Semaine 3 · Intégration

- Service Django
- Keycloak, passerelle, e-mails
- Premières pages Angular

Résultat attendu

Première version intégrée

Semaine 4 · Finalisation

- Tests d'intégration
- Docker Compose
- Documentation, démo

Résultat attendu

Version de démonstration

Parcours de démonstration prioritaire

S'authentifier → Consulter la météo → Afficher les prévisions → Configurer une alerte → Recevoir l'e-mail

Principaux risques

Risque	Conséquence	Solution prévue
Retard d'un service	Retard d'intégration	Livrables intermédiaires, suivi hebdomadaire
Incompatibilité entre API	Échec des échanges	Contrats REST, schémas JSON et codes HTTP fixés à l'avance
API météo indisponible ou quotas dépassés	Données manquantes	Cache, données de test, gestion des erreurs
Accès croisé aux bases	Couplage fort	Interdiction d'accès direct, passage par les API
Mauvaise configuration de Keycloak	Accès refusés ou trop permissifs	Realm versionné, rôles documentés, tests d'autorisation
Conflits Git et intégration difficile	Régressions, retards	Branches dédiées, Pull Requests, Docker Compose progressif
Échec d'envoi d'un e-mail	Utilisateur non informé	Statut dans alert_db, journalisation, nouvelle tentative

Conclusion

Modularité

Sept services aux responsabilités claires

Sécurité

Keycloak, JWT et rôles de bout en bout

Faible couplage

Une base par service, échanges REST

Évolutivité

SMS, push, événements, Eureka, Config Server

Merci

Des questions ?

